Mission Control Academy — Implementation Report

Educational orbit-determination simulator · release 1.0.0 · 2026-11-01

This report is self-contained: what was built, how to run it, the choices made and why, the evidence that it works, and what remains. The chronological log is in `docs/BUILD_JOURNAL.md`; the detailed equations are in `docs/MATH_AND_UNITS.md`.

---

1. Summary

Location: `EduPhysics/orbital-simulator/` (new directory) plus one CI file, `EduPhysics/.github/workflows/orbital-simulator-ci.yml`. **No existing file was modified, moved, formatted or executed.** The Python reference in `EduPhysics/Python_Orbit_determination/` still carries its original timestamps, and no Python is called by the app or the build.

What it is: a browser game-like "mission control" where students fly seven missions (First Fix → Filter Showdown), plus a sandbox and a Field Guide. All physics and estimation run in a **C++20 core** (Eigen) compiled to **WebAssembly** and executed in a Web Worker; the same core builds natively with a headless CLI and 70 doctest test cases.

Run it:

```
cd orbital-simulator
export PATH="$PWD/.tools/venv/bin:$PATH" && source .tools/emsdk/emsdk_env.sh   # local toolchains
cmake --preset native && cmake --build --preset native && ctest --preset native
cmake --preset wasm && cmake --build --preset wasm
cd web && npm ci && npm run dev          # http://localhost:5173
```

Status of checks (final): native CTest 10/10 · sanitizer (ASan+UBSan) CTest 10/10 · WASM-vs-native 12/12 · Vitest 23/23 · Playwright 7/7 · ESLint 0 errors · TypeScript 0 errors · production build OK. CI workflow written and YAML-validated, **not yet run on GitHub** (the folder is not a Git repository).

---

2. Feature checklist

Legend: ✅ done and tested · ◐ done with a stated limitation · ✗ not done.

Core science

Requirement	Status	Where / note
ECI, SI, double precision, explicit epoch, stated frames	✅	`frames/`, MATH §1
Two-body gravity, circular + elliptical validation	✅	analytic circular and Kepler references
RK4 with configurable step; convergence; conservation; finite guards	✅	ratio 16.5; drift < 10^{-9}
Covariance convergence check	✅	trace plateau test; CW Riccati fixed point

Requirement	Status	Where / note
Known/unknown impulsive maneuvers; finite thrust; explicit event order	✅	`controls`, `simulation`, ARCHITECTURE "Time, nodes and event ordering"
Commanded vs actual vs measured controls	✅	`ControlModel`
12-state rigid body from the Python model; sphere inertia; thrust through CoM	✅	exact parity test with model.py equations
Euler singularity explanation and guardrails; quaternion path	✅	clamping, normalisation, warning, concept card, EXTENDING §5
6-state orbit-only mode	✅	`orbit6`
Truth/estimator separation; intentional mismatch	✅	type-enforced (`MeasurementModel`), re-filter test; estimator model overrides
EKF with variational Φ, correct Q_d time dependence, Joseph, solves, PSD checks	✅	Φ = exact RK4 Jacobian; Van Loan check
Seeded randomness, true vs assumed noise, async sensors, deterministic replay	✅	keyed RNG; action log; hash tests (native, WASM, browser)

Estimators

Requirement	Status	Note
EKF	✅	
UKF with defensible scaling and safeguards	✅	α = 1, β = 2, κ = 0 (reasoned in MATH §6)
PF: particle count, likelihood, ESS, resampling, degeneracy, seeds, cost	◐	regularised SIR; safeguards are counted and shown; process noise uses linearised Q_d
Linear KF, honestly scoped; divergence demonstration	✅	reference vs frozen; frozen diverges > 10 km; exact on CW lab
Same scenarios/metrics for fair comparison; unsupported combos explained	✅	Showdown; frozen KF refuses nonlinear sensors with reason
Plugin-like estimator addition	✅	`EstimatorRegistry`; UI lists registry

Sensors and observability

Requirement	Status	Note
GNSS position fix (≠ pseudorange); GNSS PV variant	✅	
Gyro; star tracker (quaternion-style noise → Euler)	✅	R mapped by W R W^T
Ground-station range + bearing, visibility, frames, Jacobians; range-only	✅	FD-checked; elevation mask
Toggles, cadence, σ / covariance, outage schedule, station placement	✅	cadence and outages at runtime; placement via sandbox editor
Observability lessons and numerical diagnostic (rank/conditioning), local caveat	✅	`local_observability`; attitude unobservable without thrust shown
Q/R presets, sliders, symmetric covariance editor with validation	✅	TS + C++ validation

Visualisation, controls, scoring

Requirement	Status	Note
3-D Earth/orbit, camera, spacecraft, predicted path, truth vs estimate	◐	renders (verified with software WebGL); interactive frame rate not measurable here; 2-D fallback
Play/pause/step/phase-step/speed/reset/seed/save/replay/maneuvers/thrust & torque	✅	
Per-dimension plots with units, angle wrapping; errors, bands, innovations, sensors, controls	✅	
Ellipsoids with dof-matched confidence; per-axis vs joint distinguished	✅	χ^2_3 ellipsoid, χ^2_1 bands, χ^2_2 2-D fallback
NIS and NEES with caveats	✅	caveat text under plots
Separate scores; no reward for tiny covariance	✅	symmetric calibration score

Lessons, assistant, docs, CI

Requirement	Status	Note
Declarative, validated lesson engine; event/condition rule registry	✅	zod schema; 19 global + 5 lesson rules
Seven-mission campaign with prediction → action → evidence → reasoning → debrief	✅	3 debriefs + 1 concept marked as author placeholders
Welcome screen, animated rule-based assistant, reduced motion, responsive	✅	390 px layout test; `prefers-reduced-motion` honoured
Force-model interface; third-body explanation; extension guide	✅	MATH §9, EXTENDING
README, REQUIREMENTS, ARCHITECTURE, MATH, AUTHORING, VALIDATION, journal, report	✅	
CI workflows (native, regression, frontend, WASM, browser smoke)	◐	written + YAML-valid; never executed on GitHub
PDF copy of this report	see §10	

3. Inspection of the reference code (before implementing)

The workspace held only `Python_Orbit_determination/state_estimators/` (`model.py`, `integrator.py`, `orbital_ekf.py`, `graphics.py`); there was no `.git` and no `.github`.

The 12-state dynamics and analytic Jacobian in `model.py` are correct. Every entry was checked by hand and then by finite differences in C++ (thrust direction [cψcθ, sψcθ, −sθ], Euler-rate kinematics, gravity gradient, orientation partials). The EKF's Joseph update and per-axis χ^2_1 bounds are also correct. The new implementation differs in these places (none is an error in the physics derivation):

Reference	Observation	C++ choice
`integrator.rk4`	k_4 uses $x + T\,k_3/2$ (should be $x + T\,k_3$); argument order differs from `rk_four`; not used by the EKF	single correct `rk_four`
`euler_int` docstring	says "trapezoidal", implements forward Euler	documented as forward Euler

Reference	Observation	C++ choice
`F = I + T A`	first order, inconsistent with RK4 mean propagation	variational equations with the same RK4 stages (exact discrete Jacobian)
`L Q Lᵀ`, `L = T G`	per-step variance → growth depends on step size	continuous PSD Q_c + zero-order-hold Σ_u (generalises the reference)
`inv(H P Hᵀ + R)`	explicit inverse	LDL^T solve
`R` shadowing	radius vs covariance; Joseph term reads global `R_hat` (same value here)	distinct names

Conventions reused: state ordering, ZYX Euler, short mathematical names (`x`, `u`, `u_m`, `T`, `F`, `G`, `H`, `K`, `P`), terse imperative comments, and `rk_four(f, x, u, T)`.

---

4. Architecture and key decisions

```
content/ + scenarios/ (JSON)  ─►  web/ (React, lesson engine, plots, three.js)
                                       │ Web Worker RPC (serialised)
                                       ▼
                                  osim.wasm  ◄── core/ (C++20: dynamics · integrators · frames ·
                                                        sensors · estimators · simulation ·
diagnostics)
                                                        └─► osim_cli + doctest (native)
```

#	Decision	Reason	Trade-off
1	Variational Φ/Γ/Q_d in one augmented RK4	Φ is the exact Jacobian of the step the mean uses; correct Q_d time scaling	n² extra ODEs (cheap for n ≤ 12)
2	`MeasurementModel` vs `Sensor` split	the compiler stops estimators touching truth-side noise	one extra interface
3	Node timeline incl. disabled sensors and control boundaries	truth independent of toggles; no RK4 across discontinuities; exact async sensors	more nodes than the record grid
4	Counter/time-keyed RNG (xoshiro256** + Box–Muller)	identical noise across estimators and toggles; exact replays	not bit-identical across libm implementations
5	UKF α = 1 default	α = 10^{-3} gives W_c0 ≈ -10^6 and indefinite P for orbit-scale states	σ-points at ±√n σ
6	PF safeguards (regularised kernel from pre-resampling covariance, full-rank fallback, inflated re-seed), all counted	without them the 6-D PF collapsed (NEES ~10^{11})	at a collapse it is a hybrid — reported, not hidden
7	Two labelled linear KFs	an honest KF must have a linear model; frozen shows divergence; CW lab shows exactness	—
8	JSON + packed Float64Array bridge, Web Worker	responsive UI, small transfers, no special headers	layout must stay in sync (tested)
9	Lessons as zod-validated JSON with Markdown/KaTeX	authorable without React; validation with precise messages	no executable content (by design)

#	Decision	Reason	Trade-off
10	uPlot, three.js without drei, `frameloop="demand"`	small, fast, low idle CPU	some hand-written controls code
11	WebGL fallback + error boundary	robustness on locked-down machines	2-D view is simpler
12	Calibration verdict band [1/3, 3], score dead band [0.5, 2]	single-run NEES is autocorrelated (0.34–2.1 across seeds for a consistent EKF)	coarse verdict; Monte-Carlo test is the rigorous check
13	Local toolchains in git-ignored `.tools/`	no Homebrew bottle for CMake on macOS 13; avoid touching user's conda env	~1.7 GB on disk

5. Implementation steps (milestones)

1. **Inspection and contract.** Reference review (above); frames, states, event ordering, scenario schema, estimator/sensor APIs written down first (journal entry 3).
2. **Native physics and tests.** Dynamics (orbit6, rigid12, cw6), RK4 + variational propagator, frames, scenario parser, CLI.
3. **Sensors and EKF**, then UKF, PF, linear KF, registries, simulation orchestrator with actions/replay, diagnostics (χ^2, ellipsoids, observability, run summary).
4. **Scenarios** for every mission, with ground stations placed under the computed ground track.
5. **WebAssembly** bindings and a Node integration test against native.
6. **Frontend**: worker/client, decoder, `useMission`, lesson schema/engine/content, all screens and panels, unit tests.
7. **Browser verification** with Playwright; fixes for WebGL absence and render-loop cost.
8. **Requirement re-check and refinements**: cadence, station placement, JSON editor, torque inputs, assistant hand-offs, calibration-metric robustness, bundle splitting.
9. **CI and documentation.**

Notable problems and how they were solved (details in the journal): a full disk during the first build (resolved with the user's approval by clearing pip/Homebrew download caches); EKF ≡ UKF in the first Showdown design (redesigned so nonlinearity matters before the first fix); three PF failure modes; FD round-off misread as a Jacobian error (diagnosed by step sweep); RK4's $O((n\Delta t)^2)$ relative error on Q_d (explained, documented); a test that encoded wrong physics (roll observability with body rates); a scripted edit that silently failed (caught by the re-filter test); a too-tight calibration verdict exposed by a new noise realisation.

6. Test evidence

Commands run on macOS 13.1 / Apple Clang 14.0.3 / Node 26.4 / Chrome (stable):

Command	Result
`cmake --preset native && cmake --build --preset native`	no warnings with `-Wall -Wextra -Wpedantic -Wshadow -Wconversion`
`ctest --preset native -j4`	**10/10 passed** — 70 test cases, 3 751 assertions (~8 s)
`ctest --preset native-debug -j4` (ASan + UBSan)	**10/10 passed** on the final code, no sanitizer reports (440 s)

Command	Result
`build/native/core/osim_cli --check-all scenarios`	12/12 scenarios finite
`node scripts/wasm_integration_test.mjs`	**12/12**; max relative difference 7.8×10^{-10}; hashes differ (libm)
`npm run lint` / `npm run typecheck`	0 errors / 0 errors
`npm test` (Vitest)	**23/23**
`npm run build`	OK; chunks: three 815 KB, markdown 404 KB, app 381 KB, wasm 815 KB
`npx playwright test`	**7/7** (6 app tests + WebGL render test)

Selected quantitative results (independent references):

Check	Result
Circular orbit vs analytic after one period (h = 10 s)	< 5 cm
Elliptic orbit vs Kepler solver	< 0.5 m
Energy / angular-momentum drift over 5 orbits	$< 10^{-9}$ relative
RK4 / Euler step-halving error ratio	16.52 / 1.998
Variational Φ vs FD of RK4 map; vs closed-form CW STM	$< 10^{-6}$ abs; rtol 10^{-8}
Q_d vs Van Loan; elapsed-time scaling	$< 10^{-4}$ rel; ratios 2.000, 8.000
$\chi^2_3(0.99)$	11.344867 (table 11.344867)
40-run Monte-Carlo ANEES/n (EKF, First Fix)	1.14, inside 99.9% bounds
Re-filtering logged measurements reproduces every estimate	exact
Replay of an interactive session (toggles, R, Q, burn, EKF→UKF→PF)	identical hash
Browser replay hash at 1000× vs 300× playback	identical

Showdown on the shipped seed (native):

Estimator	Tail position RMS	ANEES/n	Dynamics evaluations
EKF	≈ 9 m	≈ 1.5	1 200
UKF	≈ 10 m	≈ 0.6	16 800
Particle filter (1000)	≈ 14 m	≈ 0.5	≈ 1.2 M
Linear KF (reference)	≈ 2.1 km	≈ 5.7×10^{4} (over-confident)	1 200

---

7. Files added

All under `orbital-simulator/` unless noted (generated/build/tool folders excluded).

- **Build:** `CMakeLists.txt`, `CMakePresets.json`, `core/CMakeLists.txt`, `.gitignore`
- **C++ headers** (`core/include/osim/`): `types`, `constants`, `rng`; `dynamics/{force_model, dynamics_model, orbital_dynamics, rigid_spacecraft_dynamics, cw_dynamics}`; `integrators/{rk4, propagator}`; `frames/{frames, attitude}`; `sensors/{sensor, basic_sensors, ground_station, sensor_registry}`; `estimators/{estimator, ukf, particle_filter, linear_kf, covariance_utils,`

`estimator_registry}`; `simulation/{scenario, controls, simulation, json_util}`; `diagnostics/{statistics, ellipsoid, observability, metrics}`

- **C++ sources** (`core/src/…`) matching the headers; `core/apps/osim_cli.cpp`; `core/bindings/wasm_bindings.cpp`
- **C++ tests** (`core/tests/`): `test_{dynamics, integrators, frames, sensors, covariance, estimators, simulation, diagnostics, regression}.cpp`, `test_main.cpp`, `test_helpers.hpp`
- **Scenarios** (`scenarios/`): `first_fix`, `tracking_blackout`, `noisy_receiver`, `planned_burn`, `unexpected_burn`, `sensor_geometry_orbit`, `sensor_geometry_attitude`, `filter_showdown`, `kf_divergence`, `linear_lab_cw`, `python_reference_rigid12`, `sandbox_elliptical`
- **Content** (`content/`): `lessons/01…07-*.json`, `rules/global.json`, `bot/dialogue.json`, `concepts/core.json`
- **Scripts:** `scripts/wasm_integration_test.mjs`, `web/scripts/copy-wasm.mjs`
- **Web app** (`web/`): `package.json`, `package-lock.json`, `vite.config.ts`, `tsconfig.json`, `tailwind.config.js`, `postcss.config.js`, `eslint.config.js`, `playwright.config.ts`, `index.html`; `src/{main.tsx, App.tsx, styles.css, vite-env.d.ts}`; `src/sim/{worker, client, layout, metrics, scoring, units, linalg, covariance, types, useMission}.ts`; `src/lessons/{schema, engine, content, progress}.ts`; `src/components/{WelcomeScreen, MissionControl, LearnScreen, OrbitView, OrbitPlane2D, PlotPanel, Chart, SensorTimeline, ControlsPanel, CovarianceEditor, LessonPanel, DebriefModal, ShowdownPanel, ScenarioEditor, Bot, Markdown, ErrorBoundary}.tsx`; `src/wasm/osim.d.ts`; `tests/{content, engine, covariance, wasm}.test.ts`; `e2e/{smoke, webgl}.spec.ts`
- **Docs:** `README.md`, `docs/{REQUIREMENTS, ARCHITECTURE, MATH_AND_UNITS, LESSON_AUTHORING, VALIDATION, EXTENDING, BUILD_JOURNAL, IMPLEMENTATION_REPORT}.md`, `docs/screenshots/*.png`
- **CI (workspace root):** `EduPhysics/.github/workflows/orbital-simulator-ci.yml`

Size: ~6 600 lines of C++ (incl. tests), ~4 200 lines of TypeScript/TSX (incl. tests).

---

8. Limitations

Physics and models. Point-mass gravity only; spherical, uniformly rotating Earth; no precession, nutation or polar motion; constant mass and inertia; thrust through the centre of mass; Euler-angle attitude (guarded, not removed); GNSS as navigation fixes, not pseudoranges; the gyro's true bias is not estimated.

Estimation. The PF's process noise is Gaussian from the linearised Q_d at the mean; its safeguards make it a hybrid at collapse events (counted). UKF process noise is additive. Time-averaged NEES/NIS in the UI are correlated statistics: verdicts are coarse by design, and the rigorous Monte-Carlo test lives in the C++ suite.

Platform. Replays are bit-identical within one build; native and WebAssembly agree to $\approx 10^{-9}$ relative, not bit-for-bit. The interactive 3-D frame rate on real GPU hardware could not be measured on the development machine, which has no GPU. The 3-D view was verified to render with software WebGL, and a 2-D fallback is used automatically. Only Chrome was tested; Firefox and Safari are expected to work (standard WebAssembly/ES-module workers) but were not run.

Process. The CI workflow has not executed on GitHub, so the GCC/Linux build is untested locally (it compiled warning-free with Apple Clang under strict flags). Progress is stored per browser (`localStorage`, guarded).

---

9. Remaining work and next steps

Item	Next step
Run CI on GitHub	`git init` (or add to an existing repository), push a branch, and fix any GCC-specific issues the `native (ubuntu-latest)` job reports
GPU frame-rate check	open the app on a machine with a GPU, play First Fix at 1000×, and check that the frame rate stays smooth (Chrome DevTools → Performance)
Author placeholders	expand the debriefs of Noisy Receiver, Planned Burn and Sensor Geometry, and the `reference-frames` concept
J2 / drag / third body	follow EXTENDING §2 (interface ready, independent RAAN-drift test suggested)
Quaternion attitude	error-state EKF on a 13/12-state model (EXTENDING §5)
Batch least squares	a natural fifth estimator for OD teaching, as a registry entry
Firefox/Safari smoke tests	add Playwright projects for `firefox` and `webkit`

10. Final verification notes and PDF copy

- Markdown (authoritative): `orbital-simulator/docs/IMPLEMENTATION_REPORT.md`.
- PDF copy: `orbital-simulator/docs/IMPLEMENTATION_REPORT.pdf` (A4, 8 pages), generated from the Markdown with `cd web && node scripts/report-pdf.mjs` — the app's own Markdown/KaTeX renderer printed by headless Chrome. Regenerate after editing.
- Final sanitizer run (after the last C++ change): CTest log shows 10 × "Test Passed", no `LastTestsFailed.log`, no sanitizer output.
- Final frontend run after the last WebAssembly rebuild: Vitest 23/23, Playwright 7/7.
- Disk: your volume was nearly full before this work started. When it filled during the first build, I asked first and then, with your approval, cleared the pip and Homebrew download caches. Footprint now: `.tools/` (CMake venv + emsdk) 1.7 GB, `web/node_modules` 176 MB, `build/` 77 MB. The 0.9 GB sanitizer build was deleted after its final run (regenerate with the `native-debug` preset). Free space at the end: about 2.7 GB. Deleting `.tools/emsdk` frees 1.6 GB if you don't need to rebuild the WebAssembly module.